



AULA 2

Problemas de regressão



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Reconhecer problemas de regressão no contexto de *machine learning*.
- Diferenciar as métricas de avaliação de modelos de regressão.
- Entender o funcionamento dos principais algoritmos de regressão.
- Construir um modelo de regressão.

Que história é essa?

Falaremos também sobre as características dos problemas de regressão e veremos as principais métricas de avaliação utilizadas para modelos de regressão. Além disso, entenderemos o funcionamento dos principais algoritmos de *machine learning* utilizados em problemas de regressão e aprenderemos a construir esses modelos na prática.

Problemas de regressão

Os problemas de classificação, estudados na aula anterior, podem ser considerados um subtipo dos problemas de regressão, também conhecidos como problemas de estimação. Eles funcionam de forma

similar, com a diferença de que, em vez de o resultado ser categórico, como na classificação, é numérico, podendo ser contínuo ou discreto.



Mantendo o foco

Por exemplo, um problema de classificação seria: “Conceder ou não crédito para um cliente?”. Já um problema de regressão seria: “Conceder qual valor de crédito para um cliente?”.

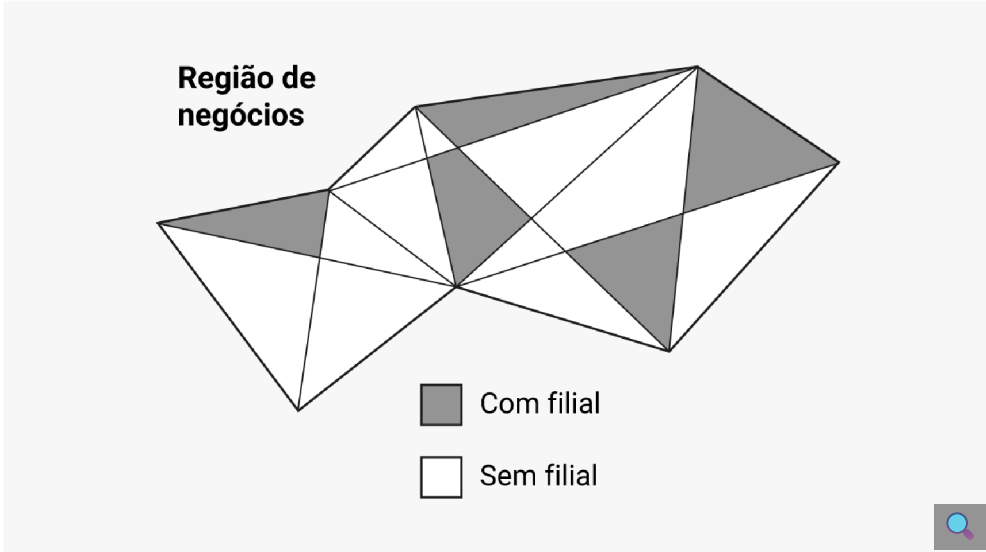
Tarefas como **preparação da base de dados, separação em conjuntos de treino e teste, definição dos critérios de parada do algoritmo, treinamento e teste** são feitas de forma equivalente para ambos os problemas.



Assim como na classificação, a regressão consiste em realizar aprendizado supervisionado a partir de dados históricos.

Além do tipo de resultado, a principal diferença entre os dois problemas está na avaliação de saída: na regressão, em vez de se estimar a acurácia, **estima-se a distância ou o erro entre a saída do estimador (modelo) e a saída desejada**. A saída de um estimador é um valor numérico contínuo, que deve ser o mais próximo possível do valor desejado, e a diferença entre esses valores fornece uma medida de erro de estimação do algoritmo.

Para entender, em linhas gerais, o que é um problema de regressão, considere um grupo varejista com esta região de negócios:



Esse grupo deseja expandir suas filiais, mas tem o seguinte problema: **“Onde abrir uma nova filial?”**. A resposta para essa pergunta pode ser determinada usando como métrica seu faturamento médio anual. Essa métrica é conhecida nas regiões onde o grupo já tem filiais, mas desconhecida onde não tem:

Bairro	Faturamento
A	105.000
B	NA
C	NA
D	NA
...	...
K	NA
L	150.000
M	NA

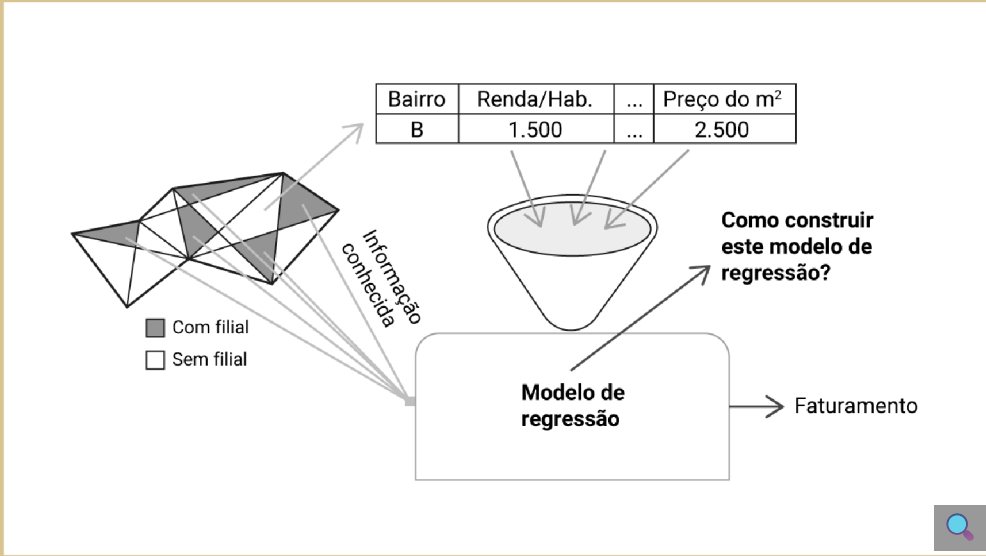
Um problema de regressão seria estimar esses valores de regiões em potencial para novas filiais. Para isso, o primeiro passo seria levantar as variáveis presentes nos bairros com e sem filiais, como:

- renda *per capita*;
- IDH;
- número de concorrentes;
- número de habitantes;
- preço do m².

A seguir, separamos nossos dados em dois conjuntos: com e sem faturamento, como ilustra a tabela:

	Bairro	Renda/Hab.	...	Preço do m ²	Faturamento
Com	A	1.500	...	2.500	105.000
	H	2.400	...	5.300	180.000
	...	...	...	...	...
	L	3.400	...	2.750	150.000
Sem	B	2.500	...	1.780	NA
	C	1.000	...	3.500	NA
	D	4.300	...	6.500	NA
	...	...	...	...	...
	K	7.000	...	8.900	NA
	M	2.800	...	3.900	NA

Com esses dados em mãos, basta elaborarmos um modelo de regressão para obtermos os valores estimados para as regiões sem filiais:



Finalmente, a partir do modelo, seleciona-se o bairro com maior faturamento esperado para abrir uma nova filial:

	Bairro	Renda/Hab.	...	Preço do m²	Faturamento
Observado	A	1.500	...	2.500	105.000
	H	2.400	...	5.300	180.000
	...	...	...	...	...
	L	3.400	...	2.750	150.000
Esperado	B	2.500	...	1.780	NA
	C	1.000	...	3.500	200.000
	D	4.300	...	6.500	NA
	...	...	...	...	...
	K	7.000	...	8.900	180.000
	M	2.800	...	3.900	120.000

Assim, podemos definir um problema de regressão como: dado um conjunto de n padrões, em que cada um é composto por informação de variáveis explicativas (independentes) e uma variável resposta contínua ou discreta (dependente), tem-se como objetivo construir um modelo de regressão que, dado um novo padrão, estime o valor **mais esperado** para a variável resposta.

A tabela a seguir ilustra o problema de regressão:

Padrão	Explicativas			Resposta
x_1	x_{11}	...	x_{1j}	y_1
x_2	x_{21}	...	x_{2j}	y_2
x_3	x_{31}	...	x_{3j}	y_3
...	...	...	...	...
x_i	x_{i1}	...	x_{ij}	y_i
...	...	...	...	...
x_n	x_{n1}	...	x_{nj}	y_n

Formalmente, seja d_j a resposta desejada para o objeto j e y_j a resposta predita do algoritmo, obtida a partir da entrada x_j . Então, $e_j = d_j - y_j$ é o erro observado na saída do sistema para o objeto j . O processo de treinamento do estimador tem por objetivo corrigir esse erro e, para isso, busca minimizar um critério (função objetivo) baseado em e_j , de maneira que os valores de y_j estejam próximos dos de d_j no sentido estatístico.

A seguir, vamos entender um pouco mais as métricas de avaliação.

Métricas de avaliação

Vale a pena relembrar o teorema “não existe almoço grátis”: não há um algoritmo de aprendizado que seja superior aos demais, quando considerados todos os problemas possíveis. A cada problema, os algoritmos disponíveis devem ser experimentados, com diversas configurações, para identificar os que obtêm melhor desempenho. Para isso, precisamos utilizar métricas de avaliação apropriadas.

Saiba mais sobre elas:

Métricas de avaliação para problemas de regressão

Existem diversas métricas de avaliação para problemas de regressão, todas buscando quantificar o erro do modelo, que representa a diferença entre a saída real e a saída do modelo. Esse erro é dado por:

$$e = y_j - \hat{y}_j$$

Uma das métricas de avaliação mais usadas é a **MSE** (*mean squared error*, ou erro quadrático médio). Quanto mais próximo de zero, melhor o modelo de regressão analisado. Essa métrica fornece uma ideia da magnitude do erro, mas não da direção deste, e sua fórmula é dada por:

$$MSE = \frac{1}{n} \sum_{j=1}^n e_j^2$$

Baseada na MSE, temos a **RMSE** (*root mean squared error*, ou raiz do erro quadrático médio), que, também, quanto menor, melhor o modelo. A raiz quadrada faz com que o erro seja medido na mesma unidade da saída do problema (variável *target*), e sua fórmula é dada por:

$$RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n e_j^2}$$

Outra métrica muito utilizada é o **coeficiente de determinação**, ou **R²** (*R-square*), que explica o quanto a variável *target* pode ser explicada pelo modelo (ou seja, o quanto *y* é explicado por *X*). Varia entre 0 e 1, e quanto mais próxima de 1, melhor o ajuste do modelo aos dados. Sua fórmula é dada por:

$$R^2 = 1 - \frac{SQE}{\sum_{j=1}^n (y_i - \bar{y})^2}$$

em que **SQE** é a soma dos quadrados dos erros (*sum of squared errors*), calculada por:

$$SQE = \sum_{j=1}^n e_j^2$$

A seguir, vamos conhecer alguns algoritmos de *machine learning* que podem ser usados para problemas de regressão.

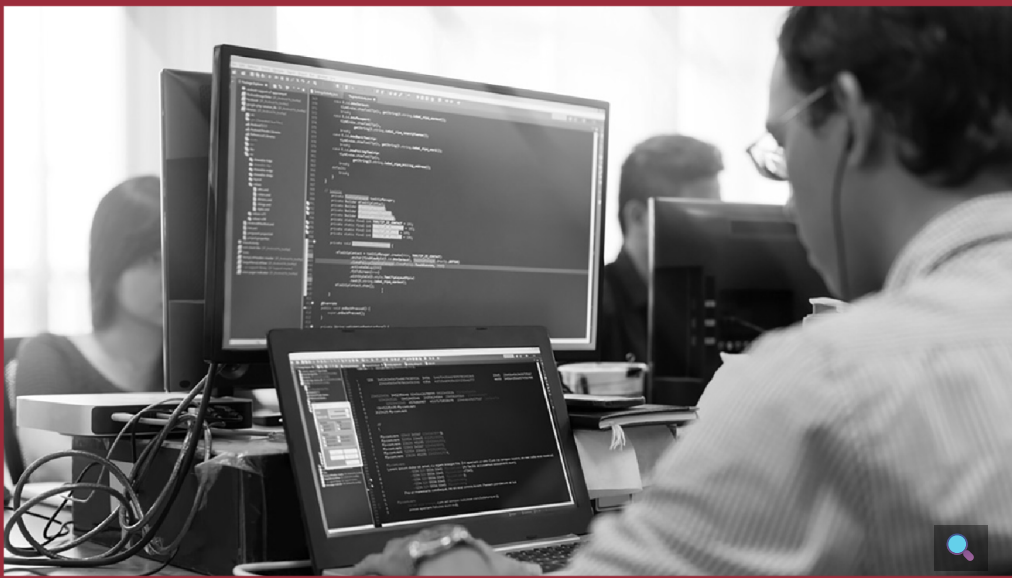
Algoritmos de *machine learning* para regressão

Antes de continuar com seu aprendizado, lembre-se de que tarefas como preparação da base de dados, separação em conjuntos de treino e teste, definição dos critérios de parada do algoritmo, treinamento e teste são feitas de forma equivalente, tanto em problemas de classificação quanto de regressão.

Árvore de decisão

Quando utilizadas para problemas de regressão, as árvores de decisão são chamadas de árvores de regressão. Foram introduzidas na década de 1980, como parte do algoritmo CART (*classification and regression tree*).

São muito similares às árvores de classificação, com a diferença de que, nas árvores de regressão, a predição (ou valor estimado) é a média dos valores dos exemplos de cada folha.



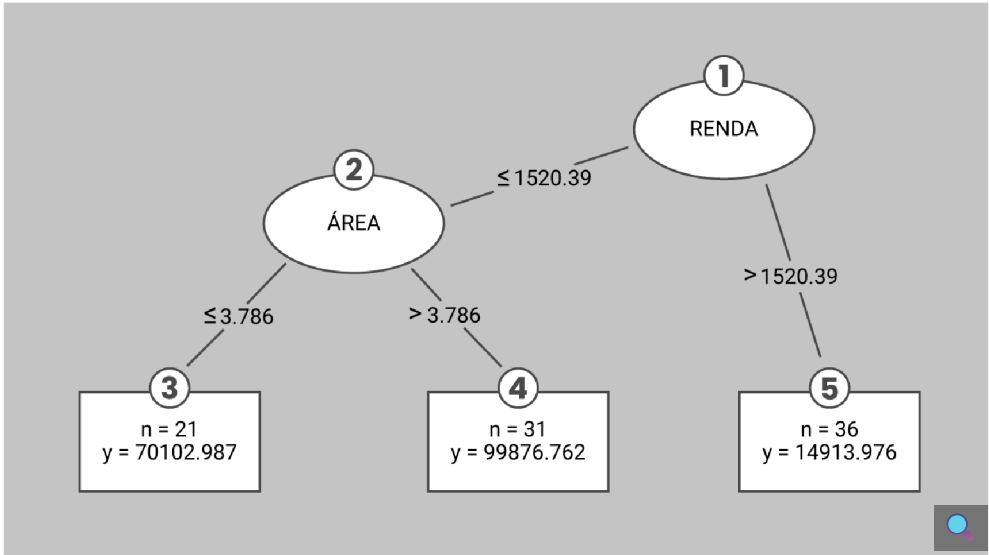
Também são construídas de forma similar às árvores de classificação: a partir do nó-raiz, os dados são particionados usando uma estratégia de divisão e conquista, de acordo com a característica que levará ao resultado mais homogêneo após a separação ser realizada. Enquanto, nas árvores de classificação, a homogeneidade é medida pela entropia, nas árvores de regressão ela é medida por **estatísticas como variância, desvio-padrão ou desvio absoluto da média**.

Um critério de divisão comum para árvores de regressão é a redução de desvio-padrão (SDR – *standard deviation reduction*).

Essa fórmula mede a redução no desvio-padrão, comparando-o antes e após a divisão.

Por exemplo, se o desvio-padrão é mais reduzido com a divisão da árvore T em uma característica B do que em A , deve-se realizar a divisão primeiro em B , resultando em uma árvore mais homogênea. Supondo que a árvore de regressão está pronta com apenas essa divisão em B , a predição pode ser feita considerando se o novo exemplo caiu no conjunto T_1 ou T_2 e calculando a média dos valores desse conjunto.

A figura a seguir ilustra uma árvore de regressão:



Na figura, 1 e 2 representam os nós, em que ocorre a divisão considerando os atributos RENDA e ÁREA, respectivamente. Os valores abaixo destes nós indicam os pontos de corte e, de acordo com o valor desta variável no exemplo avaliado, iremos para a esquerda ou para a direita. Por sua vez, 3, 4 e 5 representam as folhas, em que n representa o número de exemplos de treinamento que se encontram na folha e y representa o valor de saída para o novo exemplo que cair nesta folha.

Siga em frente para conhecer mais algoritmos.

KNN (*K-NEAREST NEIGHBOURS*)

O KNN também pode ser usado para problemas de regressão: nesse caso, o valor estimado é a média aritmética dos *k*-vizinhos mais próximos. As perguntas sobre o valor de *k* adequado e a métrica de distância a ser usada também são aplicáveis para esse tipo de problema.

SVM (*SUPPORT VECTOR MACHINE*)

O SVM também pode ser usado como método de regressão, sendo chamado de **SVR** (*support vector regression*). O SVR foi proposto em 1997, mas é pouco utilizado, pois existem modelos mais simples para regressão, com resultados semelhantes ou melhores.

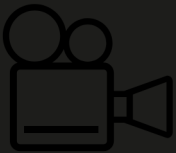
Interativo

Descrição do interativo

Comparado ao modelo de regressão linear, o SVR tem a vantagem de utilizar uma grande variedade de funções que se adequam aos modelos. Na regressão linear, tentamos minimizar a taxa de erro, enquanto, no SVR, tentamos ajustar o erro dentro de um determinado limite, definido pela margem.

Regressão linear

Vamos começar analisando um exemplo prático?



Exemplo de regressão linear

Assista ao vídeo a seguir para ver um exemplo do funcionamento da regressão linear e entender melhor como esse algoritmo funciona.



Formalmente, a regressão linear modela a relação entre a variável de resposta (y) e as variáveis preditoras (X). **A regressão corresponde ao problema de estimar uma função a partir de pares entrada-saída e considera que y pode ser explicado por uma combinação linear de X .**

Existem dois tipos de regressão linear:

Regressão linear simples	↓
Quando há apenas um x em X , temos uma regressão linear simples, cuja equação é $y = \beta_0 + \beta_1x$, sendo β_0 e β_1 os coeficientes de regressão (especificam, respectivamente, o intercepto do eixo y e a inclinação da reta).	
Regressão linear múltipla	↓
Para o caso da regressão linear múltipla, a equação deve ser estendida para a equação de plano/hiperplano: $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_nx_n$.	

Ou seja, a solução da tarefa de regressão consiste em encontrar valores para os coeficientes de regressão, de forma que a reta (ou plano/hiperplano) se ajuste aos valores assumidos pelas variáveis no conjunto de dados. Além do método dos mínimos quadrados, existem outras técnicas matemáticas para determinar os coeficientes, mas estão fora do escopo desta disciplina.



Mantendo o foco

A saída do estimador é um valor numérico contínuo que deve ser o mais próximo possível do valor desejado, e a diferença entre esses valores fornece uma medida do erro de estimação do algoritmo.

Então, seja d_j a resposta desejada para o objeto j e y_j a resposta predita do algoritmo, obtida a partir da entrada x_j . Assim, $e_j = d_j - y_j$ é o erro observado na saída do sistema para o objeto j .

O processo de treinamento do estimador tem por objetivo corrigir esse erro observado e, para isso, busca minimizar um critério (função objetivo) baseado em e_j , de maneira que os valores de y_j estejam próximos dos de d_j no sentido estatístico. Se a equação de regressão aproxima suficientemente bem os dados de treinamento, então pode ser usada para estimar o valor de uma variável (y) a partir do valor da outra variável (X), assumindo uma relação linear entre elas.

Em suma, a regressão linear procura pelos coeficientes da reta que minimizam a distância dos objetos até ela. Apesar de sua simplicidade, modelos lineares costumam ser surpreendentemente competitivos.

Métodos de regularização

A regressão linear usa o método dos mínimos quadrados para estimar seus coeficientes, buscando minimizar a soma dos quadrados dos erros (SQE – *sum of squared errors*). No caso de um coeficiente ser zero, a influência da variável de entrada no modelo é removida, pois $0 * x_i = 0$.

O modelo se torna menos complexo e com melhor interpretabilidade, pois as variáveis não relevantes (que não estão realmente associadas à resposta) são eliminadas.

Os métodos de regularização são extensões de treinamento do modelo linear que, além de buscarem minimizar a soma do erro quadrático (SQE), buscam reduzir a complexidade do modelo por meio de uma função de penalidade.

Os métodos de regularização se dividem da seguinte forma:

Regularização Ridge	↓
Busca minimizar a soma dos quadrados dos coeficientes (conhecida também como regularização L2).	
Regularização Lasso	↓
Busca minimizar a soma do valor absoluto dos coeficientes (conhecida também como regularização L1).	

Esses métodos são especialmente eficazes quando há correlação entre os atributos de entrada e os mínimos quadrados comuns superestimam os dados de treinamento, ocorrendo *overfitting*.

O parâmetro de ajuste λ serve para controlar o **impacto da penalidade**.

Quando $\lambda = 0$, o termo da penalidade não tem efeito e o resultado é similar ao método dos mínimos quadrados da regressão linear.

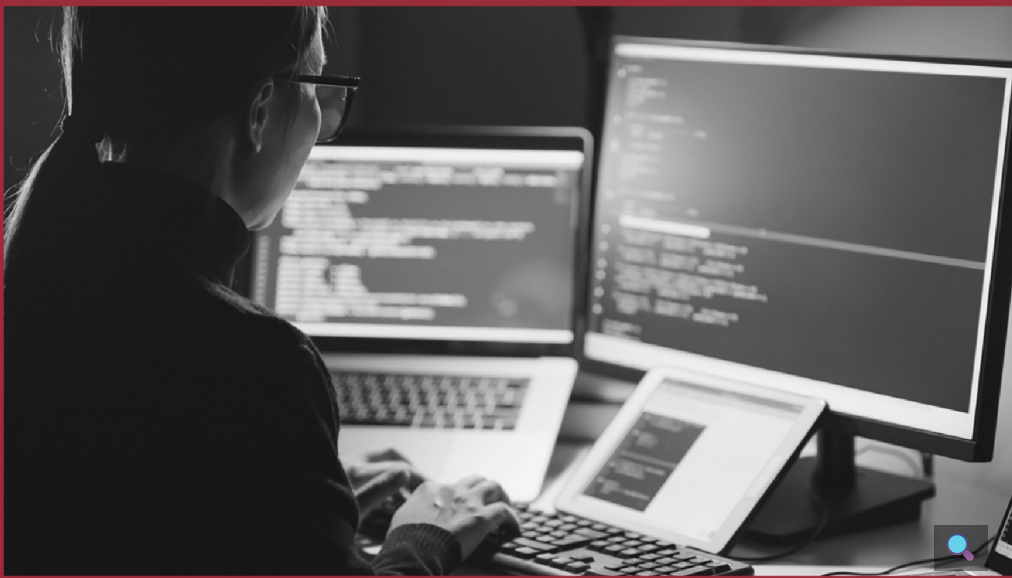
Quanto maior λ , maior o impacto da penalidade e maior a diminuição dos coeficientes.

Na regularização Ridge, a penalidade poderá diminuir todos os coeficientes para próximo de zero, mas nunca exatamente zero. O modelo gerado sempre terá todas as variáveis preditoras e não é robusto a *outliers*, podendo prejudicar a interpretabilidade do modelo, mas sendo capaz de aprender padrões mais complexos.

Já na regularização Lasso, a penalidade pode levar alguns coeficientes a exatamente zero (quando λ for suficientemente grande), realizando a seleção de variáveis preditoras e facilitando a interpretabilidade do modelo, que é mais simples e robusto a *outliers*, mas não tão capaz de aprender padrões mais complexos.

Regressão logística

A regressão logística, apesar do nome, é um algoritmo utilizado para problemas de **classificação**. Está sendo apresentada nesta aula porque seu funcionamento lembra muito o do algoritmo de regressão linear.



O algoritmo da regressão logística é usado para estimar valores discretos (valores binários como 0/1, sim/não e verdadeiro/falso) com base em um conjunto de variáveis independentes. Internamente, a regressão logística **calcula a probabilidade de ocorrência (p) de um evento**, ou seja, seus valores de saída estão entre 0 e 1, e essa saída é mapeada na classe correspondente. Assim, **a regressão logística é análoga à regressão linear múltipla** (desejamos modelar y como uma função linear de X), mas a saída é uma classe.

Acompanhe um exemplo de regressão logística:

Regressão logística

Se utilizássemos a fórmula da regressão linear $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$, não garantiríamos que a saída ficasse entre 0 e 1. Assim, modelamos a saída utilizando uma **função logística** (também chamada de **sigmoide**), uma curva em forma de S que pode mapear qualquer número em um intervalo entre 0 e 1:

$$f(x) = \frac{1}{1 + e^{-x}}$$

em que $f(x)$ é a saída prevista. Veja o exemplo de uma transformação logística entre -6 e 6:

Ou seja, de forma similar à regressão linear, a regressão logística usa uma equação como representação, em que os valores de entrada (X) são combinados linearmente, por meio de coeficientes, para prever um valor de saída (y):

$$\hat{y} = p = \frac{1}{1 + e^{-(b_0 + b_1 x_1)}}$$

O valor de saída (entre 0 e 1) é, então, arredondado para um valor binário (0 ou 1) e mapeado na classe correspondente (se for maior que 0,5, a classe é 1; caso contrário, a classe é 0).



Os coeficientes da regressão logística podem ser estimados usando os dados de treinamento, por meio do método de **estimação de máxima verossimilhança**, um método matemático que busca valores para os coeficientes de forma a minimizar o erro nas probabilidades preditas pelo modelo para os dados.

Os melhores coeficientes resultarão em um modelo que vai ter como saída um valor muito próximo de 1 para a classe-padrão, e um valor muito próximo de 0 para a outra classe. Após determinados os coeficientes, para fazer previsões com a regressão logística, basta aplicar a equação resultante.

Acompanhe um exemplo de regressão em *machine learning* utilizando Python.

Prática de *machine learning* em Python (regressão)

Para ilustrar como aplicamos os algoritmos de *machine learning* de regressão na prática, vamos examinar o *dataset* Diabetes (para regressão).

Esse *dataset* contém 10 variáveis de linha de base sobre 442 pacientes com diabetes:

Interativo

Descrição do interativo

A variável *target* é uma medida quantitativa da progressão da doença um ano após a linha de base, ou seja, é um problema de regressão.

Em seguida, para a base de treino, vamos avaliar o MSE e o RMSE dos modelos treinados com os algoritmos:

- regressão linear;
- regressão linear com regularização Ridge;
- regressão linear com regularização Lasso;
- KNN;
- árvore de regressão;
- SVM.

Utilizaremos sua configuração-padrão da biblioteca scikit-learn, ou seja, sem variar seus hiperparâmetros. Para uma melhor avaliação, utilizaremos o método de validação cruzada (10 *folds*) e compararemos os resultados graficamente por meio de *boxplots*.

Todo o código está comentado, para facilitar o entendimento. Iniciaremos esta prática importando os pacotes necessários para o *notebook*:

```
1 # Configuração para não exibir os warnings
2 import warnings
3 warnings.filterwarnings("ignore")
4
5 # Imports necessários
6 import pandas as pd
7 import numpy as np
8 import matplotlib.pyplot as plt
9 from sklearn.datasets import load_diabetes # para importar o dataset diabetes
10 from sklearn.model_selection import train_test_split # para particionar em bases de treino e teste (holdout)
11 from sklearn.model_selection import KFold # para preparar os folds da validação cruzada
12 from sklearn.model_selection import cross_val_score # para executar a validação cruzada
13 from sklearn.metrics import mean_squared_error # métrica de avaliação MSE
14 from sklearn.linear_model import LinearRegression # algoritmo Regressão Linear
15 from sklearn.linear_model import Ridge # algoritmo Regularização Ridge
16 from sklearn.linear_model import Lasso # algoritmo Regularização Lasso
17 from sklearn.neighbors import KNeighborsRegressor # algoritmo KNN
18 from sklearn.tree import DecisionTreeRegressor # algoritmo Árvore de Regressão
19 from sklearn.svm import SVR # algoritmo SVM
```

A seguir, iremos preparar os dados. Primeiro, carregaremos o *dataset* a partir da biblioteca scikit-learn. Depois, aplicaremos o *holdout* para efetuar a divisão em bases de treino (80%) e teste (20%). Por fim, separaremos em 10 *folds*, usando a validação cruzada.

```
1 # Carga do dataset
2
3 diabetes = load_diabetes()
4 dataset = pd.DataFrame(diabetes.data, columns=diabetes.feature_names) # conversão para dataframe
5 dataset['target'] = diabetes.target # adição da coluna target
6
7 dataset.head()
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019908	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068330	-0.092204	75.0
2	0.083299	0.050680	0.044451	-0.005671	-0.045509	-0.034194	-0.032356	-0.002592	0.002864	-0.025930	141.0
3	-0.089063	-0.044642	-0.011995	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022692	-0.009362	206.0
4	0.003383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031991	-0.046641	135.0

```
1 # Preparação dos dados
2
3 # Separação em bases de treino e teste (holdout)
4 array = dataset.values
5 X = array[:,0:10] # atributos
6 y = array[:,10] # classe (target)
7
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=7) # faz a divisão
9
10 # Criando os folds para a validação cruzada
11 num_particoes = 10 # número de folds da validação cruzada
12 kfold = KFold(n_splits=num_particoes, shuffle=True, random_state=7) # faz o particionamento em 10 folds
```

Em seguida, passaremos para a etapa de modelagem. Definiremos uma semente global para essa célula de código (a semente é necessária para garantir a reprodutibilidade do código, com os mesmos resultados) e criaremos os modelos com os algoritmos regressão linear, regressão linear com regularização Ridge, regressão linear com regularização Lasso, KNN, árvore de regressão e SVM, adicionando-os em uma lista.

Depois, cada um destes modelos será treinado e avaliado com a base de treino, usando a validação cruzada 10-fold. O resultado médio da acurácia de cada modelo será impresso, bem como um gráfico *boxplot* resumizando os resultados das 10 execuções (correspondentes aos 10 *folds*):

```
1 # Modelagem
2
3 # Definindo uma seed global para esta célula de código
4 np.random.seed(7)
5
6 # Listas para armazenar os modelos, os resultados e os nomes dos modelos
7 models = []
8 results = []
9 names = []
10
11 # Preparando os modelos e adicionando-os em uma lista
12 models.append(('LR', LinearRegression()))
13 models.append(('Ridge', Ridge()))
14 models.append(('Lasso', Lasso()))
15 models.append(('KNN', KNeighborsRegressor()))
16 models.append(('CART', DecisionTreeRegressor()))
17 models.append(('SVM', SVR()))
18
19 # Avaliando um modelo por vez
20 for name, model in models:
21     cv_results = cross_val_score(model, X_train, y_train, cv=kfold, scoring='neg_mean_squared_error')
22     results.append(cv_results)
23     names.append(name)
24     # Imprime MSE, desvio padrão do MSE e RMSE dos 10 resultados da validação cruzada
25     msg = "%s: MSE %0.2f (%0.2f) - RMSE %0.2f" % (name, abs(cv_results.mean()), cv_results.std(), np.sqrt(abs(cv_results.mean())))
26     print(msg)
27
28 # Boxplot de comparação dos modelos
29 fig = plt.figure()
30 fig.suptitle('Comparação do MSE dos Modelos')
31 ax = fig.add_subplot(111)
32 plt.boxplot(results)
33 ax.set_xticklabels(names)
34 plt.show()
```

A saída resultante será:

LR: MSE 3066.48 (612.06) - RMSE 55.38
Ridge: MSE 3566.43 (805.55) - RMSE 59.72
Lasso: MSE 3948.90 (891.00) - RMSE 62.84
KNN: MSE 3522.14 (721.76) - RMSE 59.35
CART: MSE 6431.26 (1584.05) - RMSE 80.20
SVM: MSE 5285.08 (1186.19) - RMSE 72.70

No scikit-learn, o MSE e o RMSE são exibidos como valores negativos. Como imprimimos seu valor absoluto, neste caso, quanto menor o valor impresso, melhor é o resultado. Analisando os resultados, verificamos que, considerando o MSE (e, consequentemente, o RMSE), o modelo treinado com a regressão linear apresentou os melhores resultados, indicando que, possivelmente, seguiríamos com ela como escolha de algoritmo. Nesse caso, construiremos um novo modelo, treinado com toda a base de treino.

Este modelo será avaliado utilizando a base de teste:

```
1 # Criando um modelo com todo o conjunto de treino
2 model = LinearRegression()
3 model.fit(X_train, y_train)
4
5 # Fazendo as predições com o conjunto de teste
6 predictions = model.predict(X_test)
7
8 # Estimando o MSE e o RMSE no conjunto de teste
9 mse = mean_squared_error(y_test, predictions)
10 print("MSE %0.2f" % mse)
11 print("RMSE %0.2f" % np.sqrt(abs(mse)))
```

MSE 2965.05
RMSE 54.46



Fique ligado!

Ressaltamos que esse é um exemplo simples, apenas para ilustrar a aplicação de algoritmos de *machine learning* em problemas de regressão.

Em problemas reais, com dados mais “sujeos” e complexos, iríamos provavelmente trabalhar com operações de pré-processamento de dados (como normalização e padronização para dados quantitativos, e *One-Hot-Encoding* para dados qualitativos nominais), além de experimentar variar os hiperparâmetros dos algoritmos, o que resultaria em um código mais complexo, como veremos nas próximas aulas.



Reflita

Com base no que estudamos nesta aula, quais problemas reais, encontrados nas empresas, poderiam ser resolvidos com o apoio de modelos de *machine learning* de regressão?



Referências

Clique aqui para acessar as referências e créditos desta aula.



BISHOP, C. M.; NASRABADI, N. M. **Pattern recognition and machine learning**. New York: Springer, 2006.

BURKOV, A. **The hundred-page machine learning book**. Québec City: Andriy Burkov, 2019.

ESCOVEDO, T.; KOSHIYAMA, A. **Introdução a *data science***: algoritmos de *machine learning* e métodos de análise. São Paulo: Casa do Código, 2020.

GÉRON, A. **Mãos à obra**: aprendizado de máquina com scikit-learn, Keras & TensorFlow – conceitos, ferramentas e técnicas para a construção de sistemas inteligentes. Sebastopol: O’Reilly Media, 2021.

KALINOWSKI, M.; ESCOVEDO, T.; VILLAMIZAR, H.; LOPES, H. **Engenharia de software para ciência de dados**: um guia de boas práticas com ênfase na construção de sistemas de *machine learning* em python. São Paulo: Casa do Código, 2023.

MURPHY, K. P. **Machine learning**: a probabilistic perspective. Cambridge: MIT Press, 2012.

VANDERPLAS, J. **Python data science handbook**: essential tools for working with data. Sebastopol: O’Reilly Media, 2016.

Bancos de imagens Pexels, Pixabay, Envato, Freepik e Shutterstock.